SmartNote AI

Project explained in simple language | Evaluation and viva preparation guide

Based on the project source code

START HERE

This guide explains what the application does, how its parts work together, what the code files are responsible for, and how to answer common evaluation questions. Learn the flow first, then use the viva section to practise explaining details.

PROJECT OVERVIEW

SmartNote AI is a web application that helps a student turn lecture PDFs into study material. It extracts text from a PDF, presents the pages in a browser, and can prepare notes, important topics, practice questions, and a chat assistant.

The project uses a Python server for processing and a browser interface for interacting with the result. Gemini is an optional cloud AI service for generating study content. Some document-analysis features have local fallback logic when Gemini is unavailable.

What problem does it solve?

Students often have long lecture documents and limited time to organize them. SmartNote automates first-draft summaries and revision materials, and gives the student a place to ask questions.

One-sentence summary

SmartNote AI is a FastAPI-based study assistant that extracts lecture PDF text and turns it into browser-based notes, topic maps, practice material, and Gemini chat.

HOW THE WHOLE PROJECT WORKS

Think of the system as four stages: upload, extract, generate, and display.

- 1. Upload: the student selects a PDF in the web page. The browser sends it to the FastAPI server.
- 2. Extract: PyMuPDF reads each page and extracts text blocks. The processor records page numbers, headings, word counts, and the total text.
- 3. Generate: the AI service uses Gemini when available. Notes, topics, and practice features have local heuristic fallbacks. Chat uses Gemini directly.
- 4. Display: the API returns JSON. JavaScript updates the page with notes, topic cards, a mindmap, practice activities, or the chat reply.

Simple architecture

```
Student browser -> static HTML/CSS/JavaScript -> FastAPI routes
                                  |
                  PDFProcessor -> extracted text/pages/chunks
                                  |
                  AIService -> Gemini or local fallbacks
                                  |
                  JSON response -> browser display
```

The browser and server communicate using HTTP requests. Most requests and responses use JSON; PDF upload uses a multipart file request.

TECHNOLOGIES USED

- Python: main programming language for the server and document-processing logic.
- FastAPI: defines the HTTP API endpoints and serves the web application.
- PyMuPDF (fitz): opens PDFs and extracts text page by page.
- Google GenAI SDK: sends prompts to the configured Gemini model.
- scikit-learn and NumPy: used by the TF-IDF retrieval engine.
- HTML: defines the page structure; CSS styles it; JavaScript handles user interaction and API calls.
- Pydantic: checks the shape of incoming API request data.
- python-dotenv: reads local settings such as the API key from a .env file.

Why separate the project into services?

Each part has a clear job. main.py handles web requests, pdf_processor.py handles PDFs, ai_service.py creates AI study content, and rag_engine.py handles text retrieval experiments. This makes the code easier to understand and test than putting every task in one file.

PDF PROCESSING: FROM PAGES TO TEXT

The PDFProcessor in services/pdf_processor.py opens a file, checks its pages, and extracts text blocks. For each page it stores the page number, text, blocks, headings, and word and character counts.

The processor also creates a document title from PDF metadata or, if that is missing, from the file name. It calculates an approximate reading time using about 200 words per minute.

What is a chunk?

A chunk is a smaller piece of extracted text. The processor groups paragraphs into pieces of roughly 800 characters and overlaps the end of one piece with the beginning of the next. Overlap helps avoid losing context when a concept crosses a chunk boundary. Each chunk has an ID, page number, and text.

If the PDF is a scan containing only pictures, ordinary text extraction may find little or no text. This project does not perform optical character recognition (OCR), so scanned pages may need OCR before useful notes can be generated.

Why preserve page numbers?

Page metadata helps show where extracted content came from and makes it easier to relate study material back to the original lecture. In the current implementation, the direct Gemini chat does not use these chunks or cite their page numbers.

AI FEATURES AND FALLBACKS

Structured notes

AIService.generate_structured_notes sends the lecture text, title, and page information to Gemini and asks for a study guide with summaries, concepts, definitions, exam traps, and a revision sheet. The Gemini prompt limits the text sent to about 350,000 characters. If Gemini is unavailable or fails, a local structural note generator is used.

Important topics

AIService.extract_important_topics asks Gemini to return topics, priority levels, descriptions, page references, prerequisites, and mindmap data in JSON. It limits input to about 300,000 characters. If the call or JSON parsing fails, a local heuristic topic extractor is used.

Practice questions

AIService.generate_practice_questions requests multiple-choice questions, flashcards, and short-answer questions from Gemini, including answers and page references. The input is limited to about 300,000 characters. If Gemini fails, local heuristic generation supplies practice content.

Chat question and answer

AIService.answer_question sends the latest student question and up to eight recent chat messages to Gemini. It is not limited to the uploaded PDF. It returns an answer and suggested follow-up questions. It returns no PDF citations. If no Gemini key is configured, or the Gemini request fails, the API reports an error instead of pretending an offline answer came from Gemini.

What does fallback mean here?

A fallback is a simpler local method used when a cloud AI request cannot complete. It can keep notes, topics, and practice features available, but its results may be less flexible than Gemini. Chat currently has no local fallback.

RAG: IMPORTANT EVALUATION DISTINCTION

RAG means Retrieval-Augmented Generation. A typical RAG system first searches a document for relevant passages, then gives those passages to a language model to help answer a question.

This project contains a RAGEngine in services/rag_engine.py. It converts document chunks into TF-IDF vectors and uses cosine similarity to rank chunks against a search query. TF-IDF gives more weight to terms that are informative within the document; cosine similarity compares the direction of the query vector with each chunk vector.

However, the current production chat route does not call rag_engine.search or build a PDF context prompt. It sends the chat question and conversation history directly to Gemini. PDF extraction still creates chunks, but the active chat answer is general Gemini chat, not PDF-grounded RAG.

The system test separately indexes a sample document and checks that retrieval works. That proves the retrieval component can be tested; it does not mean the live chat endpoint is using retrieval.

Short evaluation answer

The project has a TF-IDF and cosine-similarity RAG component, but the current chat feature is not RAG. It calls Gemini directly without retrieving PDF excerpts.

MAIN SOURCE FILES

- main.py: creates the FastAPI application, request models, routes, document storage, startup cleanup, and static-file serving.
- services/pdf_processor.py: extracts text and metadata from PDF pages and creates text chunks.
- services/ai_service.py: generates notes, topics, and practice using Gemini or local fallbacks; handles direct Gemini chat.
- services/rag_engine.py: indexes chunks with TF-IDF and retrieves similar chunks with cosine similarity.
- static/index.html: defines the user interface layout and feature tabs.
- static/app.js: handles uploads, calls API routes, updates the interface, and stores chat history in browser memory.
- static/style.css: controls colors, layout, responsive styling, and animations.
- config.py: loads the .env file, API key, model name, port, and local data folders.
- test_system.py: runs component-level and end-to-end-style checks, including PDF extraction, RAG retrieval, fallbacks, and mocked Gemini chat.
- test_api.py: exercises HTTP routes through FastAPI TestClient and mocks Gemini for chat tests.

- .env: local configuration for the key and model. It should stay private and must not be committed or shared.

IMPORTANT API ROUTES

- GET /api/status: reports whether a Gemini key is configured and which model is selected.
- POST /api/config/key: accepts a key through the settings interface and can save it into .env.
- POST /api/upload: accepts a PDF, extracts its text, and stores document data.
- GET /api/document/{doc_id}: returns the document overview and generated materials.
- POST /api/generate-notes/{doc_id}: creates structured notes.
- POST /api/generate-topics/{doc_id}: creates prioritized topics and mindmap data.
- POST /api/generate-practice/{doc_id}: creates quizzes, flashcards, and short-answer questions.
- POST /api/generate-all/{doc_id}: runs notes, topics, and practice generation sequentially.
- POST /api/chat: sends a question and chat history to Gemini; it does not require a document ID.
- GET /api/export/{doc_id}/markdown and /html: export generated notes as Markdown or printable HTML.

The {doc_id} part is a placeholder for the ID returned after uploading a document. The API returns status codes: for example, 200 means success, 404 means a requested resource was not found, 503 means chat is not configured, and 502 means the Gemini request failed.

CONFIGURATION, STORAGE, AND LIFECYCLE

config.py loads the local .env file and reads GEMINI_API_KEY, DEFAULT_MODEL, and PORT. The .env file is ignored by Git in this project, which helps keep secrets out of source control. Never paste an API key into screenshots, chats, or a public repository. If a key has been exposed, revoke it and create a replacement.

Uploaded PDFs are written to uploads/ and parsed document data is written to data/ as JSON. The app also keeps active documents in a Python dictionary in memory.

Important lifecycle behavior: on server startup, main.py clears saved PDFs and JSON files. Therefore documents are not restored across a server restart. This is a current implementation detail, not durable long-term storage.

The browser keeps chat history in JavaScript memory and sends a small recent slice with each new question. Refreshing the page clears that chat history.

TESTING AND QUALITY CHECKS

Run the system test with the project Python environment: python test_system.py. It creates a sample PDF, extracts text, checks chunk retrieval, exercises note/topic/practice generation, and mocks Gemini for the direct chat check.

The chat test uses a mock response, so it checks prompt construction and API wiring without making a real Gemini request. A successful mocked test does not prove that the configured key or model is accepted by Google.

For a live integration check, confirm /api/status reports Gemini configured, verify the exact model is available to the key, then send one short chat question. Avoid sending the actual key in logs or evaluation material.

What should tests cover?

- PDF extraction: correct page count, text, and chunk page labels.
- Fallbacks: notes, topics, and practice return useful shapes with no API key.
- Chat: sends user history, handles empty Gemini replies, and returns a clear error if Gemini fails.

- Routes: missing document, invalid file, successful upload, generation, and export behavior.

STRENGTHS AND LIMITATIONS

Strengths

- The code is divided into manageable parts and has an automated test script.
- Page-level extraction and chunk metadata make the source PDF easier to inspect.
- Local generation fallbacks let several study features run without a working Gemini call.
- The UI brings notes, topics, quizzes, flashcards, exports, and chat together.

Limitations to discuss honestly

- The active chat is not grounded in the uploaded PDF and does not cite its pages.
- Scanned PDFs need OCR, which is not implemented in the current processor.
- Document files are cleared at server startup, and browser chat history is not persistent.
- Cloud AI requires an available model, a valid key, network access, and available quota. A configured key alone does not guarantee successful requests.
- The one-click generate-all endpoint runs the generations one after another, which can take time.
- Generated content may be incomplete or incorrect and should be checked against the lecture.

EVALUATION / VIVA QUESTIONS AND ANSWERS

1. What is the main purpose of SmartNote AI?

It extracts lecture text from PDFs and helps students create notes, topic lists, practice questions, and AI chat answers.

2. What is FastAPI doing in this project?

It runs the backend web server, defines HTTP endpoints, validates request data, and returns results to the browser.

3. How does the project read a PDF?

PDFProcessor uses PyMuPDF to extract text blocks from each page, preserve page details, and create smaller text chunks.

4. What is the difference between the frontend and backend?

The frontend is the browser interface made from HTML, CSS, and JavaScript. The backend is the Python server that processes files and calls AI services.

5. What is a REST API endpoint?

It is a URL and HTTP method that lets the frontend request a specific operation, such as POST /api/upload to upload a PDF.

6. What is Gemini used for?

Gemini can create study notes, topics, and practice content from extracted text and answer general chat questions.

7. Does the current chat use RAG?

No. The project has a TF-IDF retrieval engine, but the live chat route sends the question and recent history directly to Gemini without PDF retrieval.

8. Explain TF-IDF and cosine similarity simply.

TF-IDF gives words a numerical importance based on how often they occur in a passage and how uncommon they are in the document. Cosine similarity compares the query vector with passage vectors to rank similar passages.

9. What is a fallback?

A fallback is a simpler local method used if a cloud AI service is unavailable. This project has local fallbacks for notes, topics, and practice, but not for chat.

10. What is the purpose of chunk overlap?

It repeats a small part of text between neighboring chunks so a sentence or idea near a boundary is less likely to lose context.

11. Where is the API key stored?

In a local .env file or through API settings. The file is ignored by Git and the key should never be committed or shared.

12. Why might chat return an error even if a key exists?

The model might not be available to the account, the key may be invalid, the network may fail, or free-tier quota/rate limits may be reached.

13. What happens after a server restart?

The startup handler deletes saved uploaded PDFs and document JSON. The app currently does not preserve uploaded documents across restarts.

14. How is chat history sent?

The browser stores recent messages in memory and sends a limited number with the next question. The server adds them to the Gemini prompt.

15. What is one improvement you would make?

I would add an optional document-grounded chat mode that retrieves relevant chunks, includes them in the Gemini prompt, and cites their pages. I would also add OCR for scanned PDFs and persistent storage.

SHORT PRESENTATION SCRIPT

Good morning. My project is SmartNote AI, a study assistant for lecture PDFs. The user uploads a PDF through the web interface. The FastAPI backend receives it, and PyMuPDF extracts text page by page. The processor also records page information and creates text chunks.

The AI service uses Gemini to generate structured notes, important topics, and practice questions. If those Gemini operations fail, the application has local heuristic fallbacks. The chat feature is different: it sends the question and recent conversation directly to Gemini, so it can answer general questions but is not currently grounded in the uploaded PDF.

The frontend is built with HTML, CSS, and JavaScript. It calls backend routes and displays the returned material. The project also has a TF-IDF and cosine-similarity retrieval component, automated tests, and Markdown or printable HTML exports. Important limitations are that scanned PDFs need OCR, documents are cleared on server startup, and live AI features depend on model access and quota.

That is the complete flow: upload, extract, generate, and display.

GLOSSARY

- API: a defined way for two software parts to communicate.
- Backend: the server-side code that handles data and application logic.
- Frontend: the part of the application the user sees and interacts with.
- Endpoint: a specific API URL for one operation.
- JSON: a text format for structured data exchanged between the browser and server.
- Chunk: a smaller piece of a longer document.
- Embedding/vector: a list of numbers representing text for comparison.
- TF-IDF: a method that scores words by their importance in a document.
- Cosine similarity: a way to compare how similar two vectors are.
- RAG: retrieval first, then generation using the retrieved information.
- Heuristic: a practical rule-based approximation, not a full language model.

- OCR: optical character recognition; converts text in images/scans into machine-readable text.
- Environment variable: a setting stored outside the main source code, such as a secret key.
- Quota/rate limit: a restriction on how many AI requests can be made in a period.

LAST-MINUTE REVISION

- Remember the flow: browser -> FastAPI -> PDFProcessor -> AIService -> response -> browser.
- Remember the file roles: main.py routes; pdf_processor.py extracts; ai_service.py generates; rag_engine.py retrieves; app.js connects the UI to the API.
- Say clearly: current chat is direct Gemini, not PDF-grounded RAG.
- Do not claim live Gemini was verified just because mocked tests pass.
- Know key limitations: no OCR, no persistence across restart, chat requires Gemini.
- Never reveal or commit the API key. Revoke a key if it has been exposed.